ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

XML with Java

Advanced Java

What is XML

- XML is a simple text-based language designed to store and transmit data in a plain text format. It stands for eXtensible Markup Language. Some salient features of XML:
  - XML is a markup language.
  - XML is a tag-based language like HTML.
  - XML tags are not predefined like HTML. You can define your own tags. That's why it's called an extensible language.
  - XML tags are designed to be self-describing.
  - XML is the W3C recommendation for data storage and transmission.

XML Parsers

- What is a parser?
  - **A program that analyses the grammatical structure of an input, with respect to a given formal grammar**
  - The parser determines how a sentence can be constructed from the grammar of the language by describing the atomic elements of the input and the relationship among them
- How should an XML parser work?

Parser - Read XML in Java

- **XML parsing refers to traversing an XML document to access or modify data.**
- **An XML parser provides a way to access or modify data in an XML document.**
- **Java provides many options for parsing XML documents:**
  - Dom Parser: Parse an XML document by loading the entire document's contents and creating its full hierarchical tree in memory.
  - SAX Parser: Parse an XML document on event-based triggers, do not load the entire complete document into memory.
  - JDOM Parser: Parse an XML document in a similar way to a DOM parser but in an easier way.
  - StAX Parser: Parse an XML document in a similar way to the SAX parser but in a more efficient way.
  - …

DOM – Document Object Model

When should DOM be used?

- You need to know a lot about the structure of an XML document.
- You need to manipulate parts of the XML document (for example, sort certain elements).
- You need to use the information in an XML document more than once
- Advantage:
  - The DOM is a generic interface for manipulating XML document structures.
  - One of its design goals is to help Java code written for a DOM-compliant parser that will run on any other DOM-compliant parser without having to make any modifications.

DOM interfaces

- DOM defines several interfaces. The most common interfaces:
  - Node: The basic data type of the DOM.
  - Element: Most of the objects you'll be dealing with are Elements. An Element is a Node.
  - Attr: Represents an attribute of an element.
  - Text: The actual content of the Element or Attr.
  - Document: Represents the entire XML document. A Document object is often called a DOM tree.

XML example

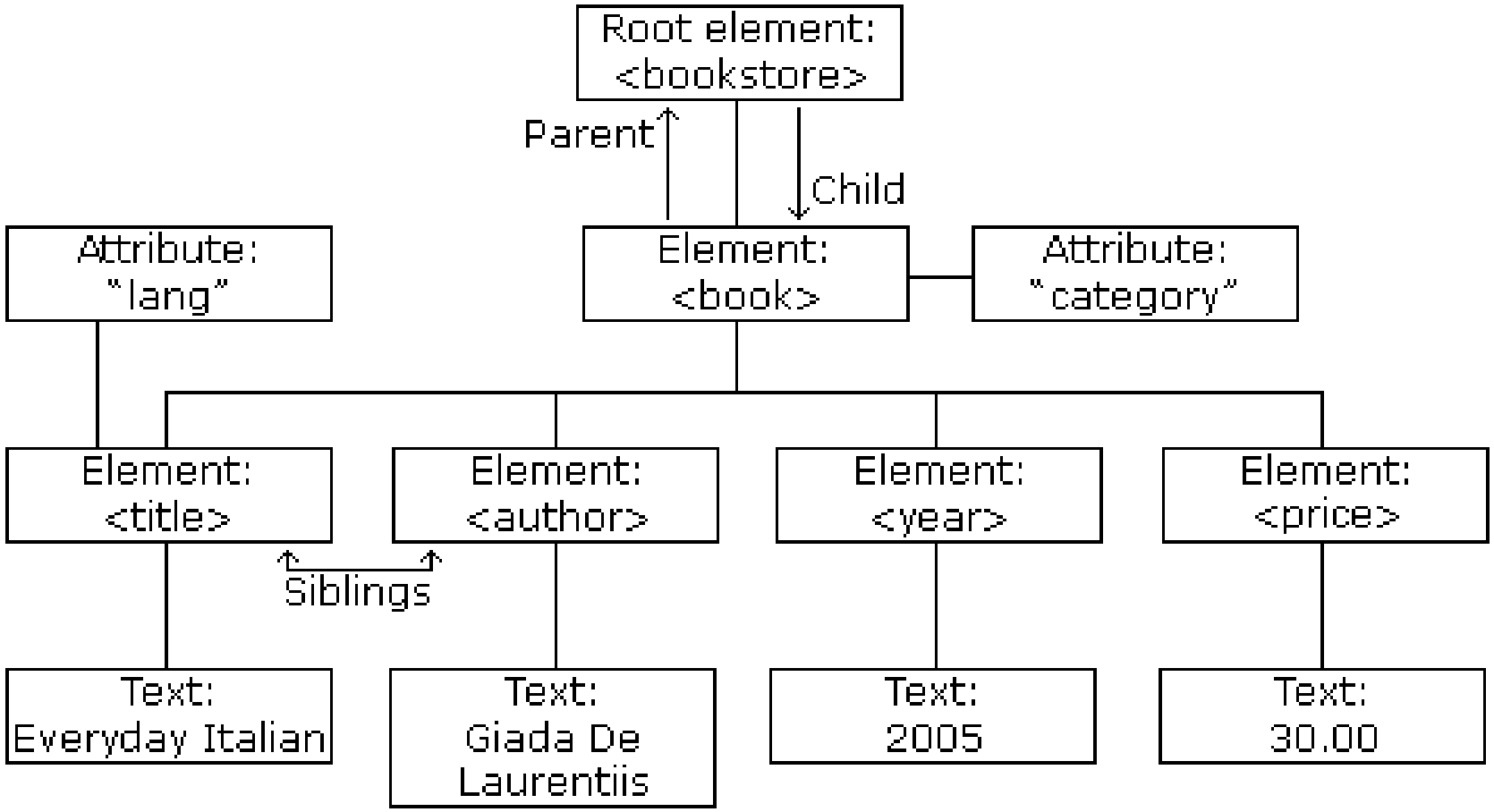
books.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
    <book category="cooking">
        <title lang="en">Everyday Italian</title>
        <author>Giada De Laurentiis</author>
        <year>2005</year>
        <price>30.00</price>
    </book>

    <book category="children">
        <title lang="en">Harry Potter</title>
        <author>J K. Rowling</author>
        <year>2005</year>
        <price>29.99</price>
    </book>
</bookstore>
```

XML tree structure

books.xml



Common DOM methods

- Document.getDocumentElement(): Returns the root element (the first child) of the document.
- Node.getFirstChild(): Returns the first child of a given Node.
- Node.getLastChild(): Returns the last child of a given Node.
- Node.getNextSibling(): This method returns the next sibling of a given Node.
- Node.getPreviousSibling(): This method returns the previous sibling of a given Node.
- Node.getAttribute(attrName): For a given Node, it returns the attribute with the requested name attrName.

Steps to use DOM

- Import XML related packages:

```
import org.w3c.dom.*;
import javax.xml.parsers.*;
import java.io.*;
```

- Create a DocumentBuilder object: 2 substeps

```
DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
```

- Create a Document object from a file or a stream:

```
Document doc = dBuilder.parse(File inputFile);
StringBuilder xmlStringBuilder = new StringBuilder();
xmlStringBuilder.append("<?xml version="1.0"?> <class> </class>");
ByteArrayInputStream input = new ByteArrayInputStream(xmlStringBuilder.toString().getBytes("UTF-8"));
Document doc = builder.parse(input);
```

Steps to use DOM (cont)

- Extract the root element:

```
Element root = doc.getDocumentElement();
//Node root = doc.getFirstChild();
```

- Check attributes:

```
//return specific attribute
Node.getAttribute("attributeName");
//return a Map (table) of name/value pairs
Node.getAttributes();
```

- Check child elements:

```
//return a list of children of the specified name
Node.getElementsByTagName("subelementName");
//return a list of all child nodes
Node.getChildNodes();
```

XML example

data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<class>
    <student id = "1">
        <name>Hoàng Oanh</name>
    </student>

    <student id = "2">
        <name>Vũ Khanh</name>
    </student>

    <student id = "3">
        <name>Khánh Ly</name>
    </student>
</class>
```

DOM - Read XML documents

- Create Student.java

```
public class Student {
    private String id;
    private String name;
    //Constructors
    //Getters and setters
    @Override
    public String toString() {
        return "Student{" + "id=" + id + ", name=" + name + '}';
    }
}
```

DOM - Read XML documents

Create a DOMReadXML.java file

```
import org.w3c.dom.*;
import javax.xml.parsers.*;
import java.io.*;
import java.util.ArrayList;
import org.xml.sax.SAXException;
public class DOMReadXML{
    public static void main(String[] args) {
        ArrayList<Student> list = DOMReadXML.readList();
        //display list of students to the screen
        for (Student student : list) {
            System.out.println(student.toString());
        }
    }
```

DOM - Read XML documents

```
public static ArrayList<Student> readList() {
    ArrayList<Student> list = new ArrayList<>();
    Student student;
    try {
        //read file data.xml
        File inputFile = new File("src\\xml01\\data.xml");
        DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
        DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
        Document doc = dBuilder.parse(inputFile);
        //print the root element to the screen
        System.out.println("Root element: " + doc.getDocumentElement().getNodeName());
        //read all elements with tag student
        NodeList nodeList = doc.getElementsByTagName("student");
```

DOM - Read XML documents

```
            //traverse the student elements
            for (int i = 0; i < nodeList.getLength(); i++) {
                student = new Student();
                //read attributes of student
                Node nNode = nodeList.item(i);
                if (nNode.getNodeType() == Node.ELEMENT_NODE) {
                    Element eElement = (Element) nNode;
                    student.setId(eElement.getAttribute("id"));
                    student.setName(eElement.getElementsByTagName("name").item(0).getTextContent());
                }
                //add student object into list of students
                list.add(student);
            }
        } catch (Exception e) {  System.err.println(e); }
        return list;
    }
}
```

The result:

```
Root element: class
Student{id=1, name=Hoàng Oanh}
Student{id=2, name=Vũ Khanh}
Student{id=3, name=Khánh Ly}
```

DOM - Create XML documents

- To create the following XML document named example.xml

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<class totalStudents="2">
    <student id="1">
        <name>Hoàng Dung</name>
    </student>

    <student id="2">
        <name>Quách Tĩnh</name>
    </student>
</class>
```

DOM - Create XML documents

Create a DOMCreateXML.java file

```
public class DOMCreateXML{
    public static void main(String argv[]){
        try {
            DocumentBuilderFactory dbFactory = DocumentBuilderFactory.newInstance();
            DocumentBuilder dBuilder = dbFactory.newDocumentBuilder();
            Document doc = dBuilder.newDocument();
            //create root element (class)
            Element rootElement = doc.createElement("class");
            doc.appendChild(rootElement);
            //add totalStudents to the class
            Attr totalStudentAttr = doc.createAttribute("totalStudents");
            totalStudentAttr.setValue("2");
            rootElement.setAttributeNode(totalStudentAttr);
            //create element student1
            Element student1 = doc.createElement("student");
            rootElement.appendChild(student1);
```

DOM - Create XML documents

```
//create id for student1
Attr attr1 = doc.createAttribute("id");
attr1.setValue("1");
student1.setAttributeNode(attr1);
//create tag name
Element name = doc.createElement("name");
name.appendChild(doc.createTextNode("Hoàng Dung"));
student1.appendChild(name);
//create element student2
Element student2 = doc.createElement("student");
rootElement.appendChild(student2);
Attr attr2 = doc.createAttribute("id");
attr2.setValue("2");
student2.setAttributeNode(attr2);
Element name2 = doc.createElement("name");
name2.appendChild(doc.createTextNode("Quách Tĩnh"));
student2.appendChild(name2);
```

DOM - Create XML documents

```
            //write to the XML file
            TransformerFactory transformerFactory = TransformerFactory.newInstance();
            Transformer transformer = transformerFactory.newTransformer();
            DOMSource source = new DOMSource(doc);
            StreamResult result = new StreamResult(new File("src\\xml01\\example.xml"));
            transformer.transform(source, result);
            //write to the console to check
            StreamResult consoleResult = new StreamResult(System.out);
            transformer.transform(source, consoleResult);
        } catch (ParserConfigurationException | TransformerException | DOMException e) {
            System.err.println(e);
        }
    }
}
```

read by DOMReadXML.java above the result:

```
Root element: class
Student{id=1, name=Hoàng Dung}
Student{id=2, name=Quách Tĩnh}
```

DOM - Modify XML documents

Edit the file example.xml above

```
public class DOMModifyXML {
    public static void main(String argv[]) {
        try {
            File inputFile = new File("src\\xml01\\example.xml");
            DocumentBuilderFactory docFactory = DocumentBuilderFactory.newInstance();
            DocumentBuilder docBuilder = docFactory.newDocumentBuilder();
            Document doc = docBuilder.parse(inputFile);
            //Extract the root element
            Node classStudent = doc.getFirstChild();//doc.getDocumentElement()
            //edit id of student1
            Node student1 = doc.getElementsByTagName("student").item(0);
            NamedNodeMap attr = student1.getAttributes();
            Node nodeAttr = attr.getNamedItem("id");
            nodeAttr.setTextContent("10");
            //edit name of student1
            student1.getChildNodes().item(0).setTextContent("Triệu Mẫn");
```

DOM - Modify XML documents

```
//delete student2
Node student2 = doc.getElementsByTagName("student").item(1);
classStudent.removeChild(student2);
//create element student3
Element student3 = doc.createElement("student");
classStudent.appendChild(student3);
Attr attr3 = doc.createAttribute("id");
attr3.setValue("3");
student3.setAttributeNode(attr3);
Element name3 = doc.createElement("name");
name3.appendChild(doc.createTextNode("Lệnh Hồ"));
student3.appendChild(name3);
```

DOM - Modify XML documents

```
            //write to the XML file
            TransformerFactory transformerFactory = TransformerFactory.newInstance();
            Transformer transformer = transformerFactory.newTransformer();
            DOMSource source = new DOMSource(doc);
            StreamResult result = new StreamResult(new File("src\\xml01\\example.xml"));
            transformer.transform(source, result);
        } catch (Exception e) {
            System.err.println(e);
        }
    }
}
```

read by DOMReadXML.java above
the result:

```
run:
Root element: class
Student{id=10, name=Triệu Mẫn}
Student{id=3, name=Lệnh Hồ}
```

SAX – Simple API for XML

SAX Parser

- SAX = Simple API for XML
- XML is read sequentially
- When a *parsing event* happens, the parser invokes the corresponding method of the corresponding handler
- The handlers are programmer's implementation of standard Java API (i.e., interfaces and classes)
- Similar to an I/O-Stream, goes in one direction

Implementing the Content Handler

- A SAX parser invokes methods such as **`startDocument`**, **`endDocument`**, **`startElement`** and **`endElement`** of its *content handler* as it runs
- In order to react to parsing events we must:
  - implement the **`ContentHandler`** interface
  - set the parser's content handler with an instance of our **`ContentHandler`** implementation
- An easy way to implement the **`ContentHandler`** interface is to extend **`DefaultHandler`**

When should SAX be used?

- SAX parser does not load the complete XML into memory
- It parses the XML by firing different events and when it encounters different elements like: opening tag, closing tag, data character, comment etc (an event-based parser)
- To read XML documents with SAX Parser we need to create a class that extends the DefaultHandler class.
- The DefaultHandler class provides the following different methods:
  - **startElement()**: trigger this event when an opening tag is encountered.
  - **endElement()**: fires this event when a closing tag is encountered.
  - **characters()**: trigger this event when it encounters some text data

XML example

data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<class>
    <student id = "1">
        <name>Vũ Khanh</name>
    </student>

    <student id = "2">
        <name>Vũ Khanh</name>
    </student>

    <student id = "3">
        <name>Khánh Ly</name>
    </student>
</class>
```

Start Document

XML example

data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<class>
    <student id = "1">
        <name>Hoàng Oanh</name>
    </student>

    <student id = "2">
        <name>Vũ Khánh</name>
    </student>

    <student id = "3">
        <name>Lê Hoàng</name>
    </student>
</class>
```

Start root Element

End root Element

XML example

data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<class>
    <student id = "2">
        <name>Hoàng Oanh</name>
    </student>

    <student id = "3">
        <name>Khánh Ly</name>
    </student>

</class>
```

End Element

Start Element

XML example

data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<class>
    <student id = "1">
        <name>Hoàng Oanh</name>
    </student>

    <student id = "2">
        <name>Vũ Khanh</name>
    </student>

    <student id = "3">
        <name>Khánh Ly</name>
    </student>
</class>
```

Attribute

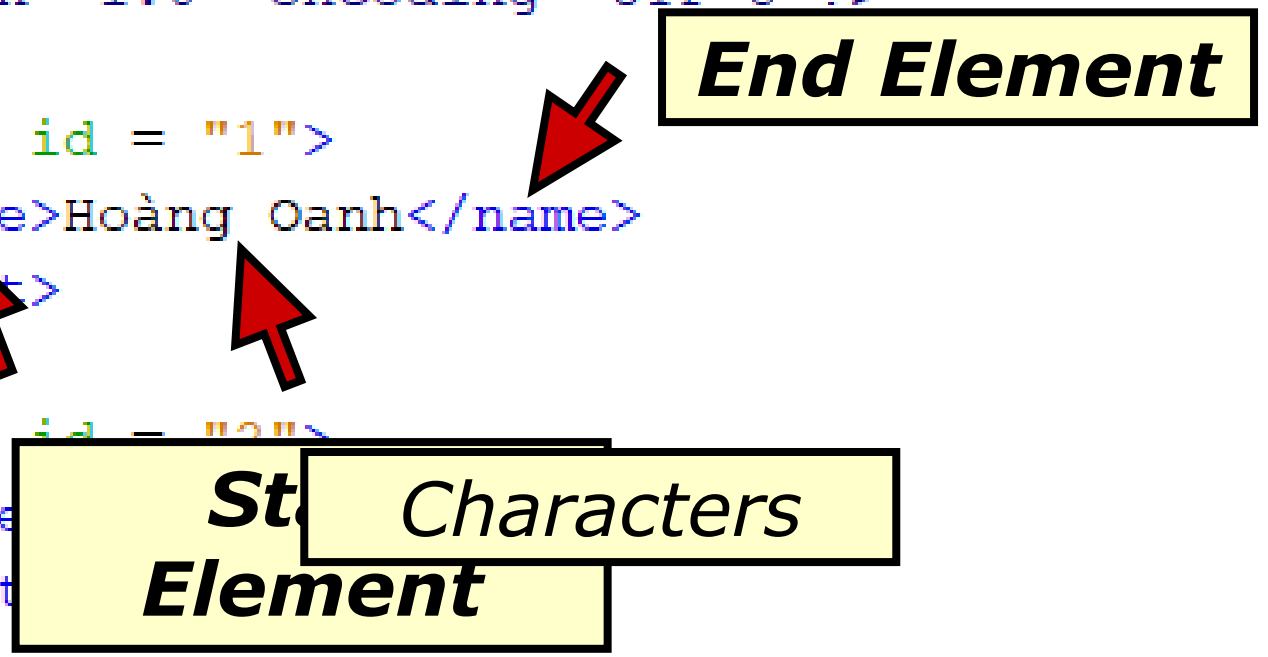
XML example

data.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<class>
    <student id = "1">
        <name>Hoàng Oanh</name>
    </student>

    <student id = "2">
        <name
    </student

    <student id = "3">
        <name>Khánh Ly</name>
    </student>
</class>
```



SAX - Read XML documents

- **Create Student.java**

```
public class Student {
    private String id;
    private String name;
    //Constructors
    //Getters and setters
    @Override
    public String toString() {
        return "Student{" + "id=" + id + ", name=" + name + '}';
    }
}
```

SAX - Read XML documents

Create a MyHandler.java file

```
public class MyHandler extends DefaultHandler {
    private String content;
    private Student student;
    private ArrayList<Student> list = new ArrayList<>();
    @Override
    public void startElement(String url, String localName, String qName, Attributes attributes)
                                                                    throws SAXException {
        //create Student object when tag "student" is encountered
        if ("student".equalsIgnoreCase(qName)) {
            student = new Student();
            student.setId(attributes.getValue("id"));
        }
    }
    @Override
    public void endElement(String url, String localName, String qName) throws SAXException {
        switch (qName) {
            case "student":
                //add Student object to list when closing tag "student" is encountered
                list.add(student);          break;
            case "name":
                student.setName(content);   break;
        }
    }
```

SAX - Read XML documents

```
    @Override
    public void characters(char ch[], int start, int length) throws SAXException {
        //read the contents of the current tag
        content = String.copyValueOf(ch, start, length).trim();
    }
    public ArrayList<Student> getList() {
        return list;
    }
    public void setList(ArrayList<Student> list) {
        this.list = list;
    }
}
```

SAX - Read XML documents: main method

```
public class SAXReadXML {
    public static void main(String[] args) {
        try {
            File inputFile = new File("src\\xml01\\data.xml");
            SAXParserFactory factory = SAXParserFactory.newInstance();
            SAXParser saxParser = factory.newSAXParser();
            MyHandler handler = new MyHandler();
            //parse the XML document
            saxParser.parse(inputFile, handler);
            //write list of student objects to screen
            for (Student student : handler.getList()) {
                System.out.println(student.toString());
            }
        } catch (IOException | ParserConfigurationException | SAXException e) {
            System.err.println(e);
        }
    }
}
```

The result:

```
Student{id=1, name=Hoàng Oanh}
Student{id=2, name=Vũ Khanh}
Student{id=3, name=Khánh Ly}
```

SAX vs. DOM

Parser Efficiency

- The DOM object built by DOM parsers is usually complicated and requires more memory storage than the XML file itself
  - A lot of time is spent on construction before use
  - For some very large documents, this may be impractical
- SAX parsers store only local information that is encountered during the serial traversal
- Hence, programming with SAX parsers is, in general, more efficient

Node Navigation

- SAX parsers do not provide access to elements other than the one currently visited in the serial (DFS) traversal of the document
- In particular,
  - They do not read backwards
  - They do not enable access to elements by ID or name
- DOM parsers enable any traversal method
- Hence, using DOM parsers is usually more comfortable

More DOM Advantages

- DOM object ⇔ compiled XML
- You can save time and effort if you send and receive DOM objects instead of XML files
  - But, DOM object are generally larger than the source
- DOM parsers provide a natural integration of XML reading and manipulating
  - e.g., “cut and paste” of XML fragments

Which should we use? DOM vs. SAX

- If your document is very large and you only need a few elements – use SAX
- If you need to manipulate (i.e., change) the XML – use DOM
- If you need to access the XML many times – use DOM (assuming the file is not too large)

Thank you!